UNIT V: Machine-Independent Optimization

Students usually leave this unit. Just study these two questions to be safe.

Long Answer (10 Marks):

1. **Crucial:** Explain the **Principal Sources of Optimization** (Function preserving transformations).
2. Discuss **Loop Optimizations** in detail: Code Motion, Induction Variable Elimination, and Strength Reduction. 3. Explain **Data Flow Analysis** and the Reaching Definitions equation.

Loop Optimization Techniques

→ Loop optimizations make loops run faster.

1. code motion or code movement
2. Loop invariant computations
3. Loop unrolling
4. Loop fusion

1. code motion : [moving one block to another]

→ It is a technique which moves the code outside the loop
- if it won't have any difference if it executes inside or outside loop.

Unoptimized code

```
for(i=0; i<n; i++)
{
    x = y+z;
    a = 6*i;
}
```

Optimized code

```
x = y+z;
for(i=0; i<n; i++)
{
    a = 6*i;
}
```

2. Loop-invariant computations : the stmts in the loop whose results of the computations do not changes over iteration.

ex:

```
a = 10;
b = 20;
c = 30;
for(i=0; i<5; i++)
{
    c = a+b;
    d = a-b;
    e = a*b;
    s = s+i;
}
```

⇒ (doesnot depend on loop)

after applying

```
a = 10;
b = 20;
c = 30;
c = a+b;
d = a-b;
e = a*b;
for(i=0; i<5; i++)
{
    s = s+i;
}
```

⇒ eliminate & place above loop.

3. Loop unrolling: Loop overhead can be reduced by reducing the no. of iterations & replacing the body of loop. ★⑤

ex:
```
for (i=0; i<100; i++)          ⟹   for (i=0; i<50; i++)
    add();                          {
                                        add();
                                        add();
                                    }
```

4. Loop fusion: merging

→ Merging of several loops into single loop.

→ improve performance

ex:
```
for (i=0; i<n; i++)                 ⟹   for (i=0; i<n; i++)
    a = 6*i;                                {
for (i=0; i<n; i++)                             a = 6*i;
    b = 6*i;                                    b = 6*i;
                                            }
```

UNIT-5

Machine-Independent Optimization:
- the Principal Sources of optimization
- Intro to Data-Flow Analysis
- Foundations of Data-Flow Analysis
- Constant Propagation
- Partial-Redundancy Elimination
- Loops in Flow Graphs

* Principal Sources of optimization (or) Code optimization techniques

→ Code optimization : removes unnecessary lines
- So, no. of lines reduces
- memory reduces
- So, executes faster

Techniques
1. Common Sub expression Elimination
2. Compile time Evaluation
   a. Constant Folding
   b. Constant Propogation
3. Code movement (or) Code motion
4. Dead code elimination
5. Strength reduction

1. Common Sub Expression Elimination (CSE)

→ CSE is an expression
- which appears repeatedly in the program
- which is computed previously but the value haven't changed.

→ It replaces redundant (↳ duplicate) expression

unoptimized code	optimized code
a = b+c	a = b+c
b = a-d	b = a-d
c = b+c	c = a
d = a-d	d = b
e = a+d	

2. Compile time Evaluation : Evaluation is done at compile time instead of runtime.

(a). Constant folding: Evaluate the expression & submit the result

ex: area = (22/7) * r * r → 3.14 * r * r

→ Calculate 22/7, replace with 3.14

So, area = 3.14 * r * r

(b). Constant Propogation: Here constant replaces a variable (Substituting)

ex: pi = 3.14, r = 5 ⇒ area = 3.14 * 5 * 5

area = pi * r * r

3. Code movement or code motion [moving one block to another]

→ It is a technique which moves the code outside the loop

- if it won't have any difference if it executes inside or outside loop

Unoptimized

```
for(i=0;i<n;i++)
{
    x = y+z;
    a=6*i;
}
```

Optimized code

```
x = y+z;
for (i=0;i<n;i++)
{
    a=6*i;
}
```

4. Dead-code Elimination : Eliminate those stmts which are never executed or if executed o/p is never used

ex:

Unoptimized code

```
i=0;
if(i==1)
{
    print(false);
}
```

optimized code

```
i=0
```

5. Strength Reduction

⇒ ~~It~~ eliminate those stmts wh

→ Replacing the expensive operator with cheaper operation.

*, +

↓

more expensive

ex: $b = a * 2 \implies b = a + 2$

* Data-Flow Analysis (DFA)

→ It is the process of collecting info about how values flow through a program.

→ used for optimization like:- constant propagation, copy propagation
- Dead code elimination

Basic Terms

1. Basic Block
2. Flow Graph

nodes = basic blocks
edges = control flow

→ Most machine-independent optimizations rely on DFA.

Important Concepts

1. Basic Block: Single entry & exit

→ A straight-line sequence of stmts.

2. Control Flow Graph (CFG)

Nodes → Basic blocks
edges → control flow b/w blocks

3. Data-Flow sets

GEN[B] - information generated within block B.

KILL[B] - info invalidated within block B

IN[B] - info available at entry to block.

OUT[B] - info available at exit from block.

4. Transfer Function:

$$OUT[B] = GEN[B] \cup (IN[B] - KILL[B])$$

5. Flow Direction:

- Forward DFA: from start → end
- Backward DFA: from end → start

* Foundations of Data-Flow Analysis

→ DFA relies on mathematical structures like lattices and fixed-point theory.

- Iterative Data-Flow Equations:
  - the solution is obtained by repeatedly applying transfer functions until a fixed point is reached.
- Lattices Operations